Compiscript three-address code

This describes scalar/control-flow, lexical-function, object/collection, guarded unknown-value and try/catch TAC for Entrega 2. The report records the final PDF matrix, resolved historical gaps and pending human checks. Functions use explicit calls/returns and deterministic abstract-cell activation records, including direct/mutual recursion, retained closures and inferred function aliases. Objects include inherited fields, construction, virtual methods, retained receivers and strings. Lists include references, literals, indexed reads/writes and foreach. Compilation JSON includes extended symbols and storage layouts; the IDE uses the same core and server-rendered TAC text. The separate `compiscript.Compile(source []byte)` library API generates typed, source-located TAC for all valid AST construction kinds, with typed and guarded operations, control flow, calls/returns and handlers. Existing analysis contracts remain unchanged except for the corrected optional-for-header mapping and missing-body recovery described below. The separate `compiscript-tac` command compiles this language without changing the existing analysis command's interface or exit policy.

Command quick path

Run the included numeric example (it computes the TAC for `x = 3.5`, without executing the source program):

```
go run ./cmd/compiscript-tac testdata/compiscript/tac/numeric.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/numeric.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/conditions.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/conditions.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/loops.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/loops.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/switch.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/switch.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/functions.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/functions.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/recursion.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/recursion.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/closures.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/closures.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/collections.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/collections.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/objects.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/objects.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/acceptance.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/acceptance.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/values-effects.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/values-effects.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/dynamic-exceptions.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/dynamic-exceptions.cps
go run ./cmd/compiscript-tac -h
```

The default format writes only TAC text to stdout. Rejected compilation writes located diagnostics to stderr, no TAC to stdout, and exits 1. JSON format writes the complete `CompilationReport` to stdout, including `tac: null` on rejection; its exit code is still 1 when diagnostics exist. Success exits 0, including an empty source. Invalid arguments or format exit 2; file/UTF-8/write failures exit 1; help exits 0 and prints usage to stderr. Flags precede the source path. `go run` can wrap a program's nonzero exit; build the binary when an automation needs the command's exact status. Scope snapshots remain semantic metadata; procedure frame layouts are separate code-generation metadata described below.

Compilation symbol contract (T6)

Successful `Compile` and CLI `-format json` responses add one optional top-level `codegen` object. Rejected compilation omits it entirely (`tac` remains null). Legacy Analyze scopes and previously supported TAC JSON/text are unchanged. Consumers can ignore the addition; do not infer storage from the semantic scopes themselves.

Field	JSON type	Meaning
`codegen.unit`	string	Always `"abstract-cell"`; every scalar or reference occupies one cell
`codegen.symbols`	array	All source semantic symbols, sorted by stable `s<scopeId>.<name>` identity
`symbols[].id`, `scopeId`, `symbol`	string, integer, object	Binding identity, original scope ID and unchanged semantic Symbol (name/kind/type/mutability/capture/span)
`symbols[].locations`	array	Every actual staging/retained/instance location; empty for non-storage or unemitted symbols
`locations[]`	object	`space` string, `offset` integer, `size` integer, `role` string
`symbols[].labels`	string array	Actual procedure labels, or class layout ID plus initializer label; no fake cell for a label
`codegen.spaces`	array	`id`/`kind` strings, optional integer `scopeId`, optional string `parent`, integer `cells`, and `slots` array
`spaces[].slots`	array	Existing FrameSlot shape: integer `offset`/`size`, string `role`, optional Operand `operand`, optional boolean `dynamic`
`codegen.labels`	array	`name`, `kind`, `owner` strings and source `span`; kinds: `procedure`, `control`, `classLayout`, `method`

For `let x=1+2;`, the symbol entry is (the actual `symbol` object also contains the original type, flags and source span):


```
{
  "id": "s1.x",
  "scopeId": 1,
  "symbol": {"name":"x","kind":"variable","type":{"kind":"integer","name":"integer","element":null,"par
  ams":[],"result":null},"mutable":true,"captured":false,"span":{"start":{"offset":0,"line":1,"column":
  1},"end":{"offset":10,"line":1,"column":11}}},
  "locations": [{"space":"globals","offset":0,"size":1,"role":"binding"}],
  "labels": []
}
```


The corresponding `globals` space has one binding slot; `main` has one dynamic temporary slot for `t0`. All arrays are present even when empty. Offsets are zero-based **within the named space**, never physical addresses:

- `globals`: emitted non-frame/non-environment symbols, sorted by identity. This includes main block bindings, not just source global declarations.
- `main`: temporary cells actually used by emitted main instructions, in emitter temporary-number order. Reused cells have no permanent type (`dynamic: true`).
- `frame:<procedure-label>`: copies the existing `Procedure.Frame` exactly, including continuation (abstract return address), control/global access links, optional result, receiver parameter, ordinary parameters, locals and temporaries.
- `environment:s<scopeId>`: bindings in the existing EnvironmentLayout order; `parent` records the existing lexical layout link, not an extra fabricated cell.
- `instance:<class-id>`: existing field offsets, including inherited fields. `parent` identifies the base layout; method labels map each class's effective dispatch entries to emitted procedures, including inherited constructors.

A parameter/local can have both a frame staging slot and retained environment binding. An inherited field appears in each relevant instance layout. The report lists all such locations; it does not invent one canonical physical address. Scope IDs identify layouts, not live invocations or heap instances. Strings, lists, objects and closures occupy reference cells; payload size and live address are unknown, and there is no claim of a physical ABI. The existing `Procedure.lexicalScope`/`captures` and environment parent layouts describe lexical access; the existing frame `globalAccessLink` slot is not renamed or repurposed.

KISS choice: derive this inventory from emitted operands and the existing frame, environment and class layouts, then join by semantic binding ID. No second parse, analysis or binding model is intro-

duced. Constants get storage only when emitted; procedure/control/layout labels and unemitted source symbols get no invented offset or size. Control labels include their owning `main`/frame space because their names may repeat across procedures. This is a static report, not a runtime interpreter. The architecture describes its CLI/IDE consumers.

Representation

An instruction has an opcode, destination, left operand, optional right operand, and the originating AST source span. The optional `label` field names a control-flow destination; it is omitted from numeric JSON, preserving existing field order and output. Operands distinguish literals, symbols, and virtual temporaries and retain the semantic `integer`, `float` or `boolean` type. The scalar representation extends to string/reference/null values and guarded `error`-kind values described below. Text renders an operand as `value:type`; structured JSON retains its kind. This representation is not SSA: an assignment can update a symbol or reuse a temporary. Spans locate source operations, not unique expression identities.

Instruction	Text	Meaning
Copy	`s1.x:integer = 1:integer`	Assign a value to a symbol or temporary
Add/subtract/multiply	`t0:integer = 2:integer * 3:integer`	Numeric operation
Divide	`t0:float = 6.0:float / 2.0:float`	Floating-point division
Remainder	`t0:float = 7.5:float % 2.0:float`	Truncating remainder; zero divisor faults
Print	`print s1.x:integer`	Read `left` once and append one output line; no destination
Declare	`declare s1.x:integer`	Activate/reserve a binding with an undefined cell; no default write
Negate	`t0:integer = neg s1.x:integer`	Numeric unary minus
Integer to float	`t0:float = i2f s1.x:integer`	Explicit numeric widening
Comparison	`t0:boolean = 1:integer < 2:integer`	`==`, `!=`, `<`, `<=`, `>`, `>=`; boolean equality/inequality also supported
Logical not	`t0:boolean = ! s1.ready:boolean`	Boolean negation
Label	`L0:`	Control-flow destination
Jump	`goto L0`	Unconditional edge
Branch true	`if t0:boolean goto L0`	Jump when the boolean left operand is true; otherwise fall through
Call	`t0:integer = call s1.f(2:integer)`	Evaluate caller-frame argument values, invoke the named procedure with fresh storage, resume and write its typed result
Null-result call	`call s1.update()`	Invoke without a callee result cell; an expression use yields null after effects
Return	`return t0:integer` or `return`	Transfer a checked result, or no value, to the saved caller continuation

Label/jump instructions use `label`; branch-true also uses `left`. Their unused legacy `target` operand is zero-valued; label/jump `left` is also zero-valued, and `right` is omitted. These fields are not value reads or writes. Labels start at `L0` and are allocated deterministically per instruction tape (main or procedure). Existing numeric instruction text and JSON remain unchanged.

Each instruction reads all source operands before writing its destination. Consequently, a destination may reuse the last-consumed operand's temporary. An integer denotes an abstract exact integer; target-

specific width and ABI are not selected here. Floats denote IEEE-754 binary64 values; integer widening may round. Mixed operations widen integers explicitly. Division widens integer operands and produces a float, matching the semantic analyzer. No constant folding is performed; division by zero is a runtime fault for a future backend, not permission to omit instructions. There is no product TAC executor in this unit.

Binding and lifetimes

The source compiler assigns symbol names `s<scope-id>.<source-name>` from the semantic scope inventory. These are logical identities, not byte offsets, and are deterministic for the same source, not stable across source edits. Variable declarations become visible only after their initializer is lowered. This preserves outer-variable binding in a shadowing initializer. Named function identities/signatures are predeclared for forward calls; variable bindings are not. Hoisting applies to every statement sequence, including nested bodies and switch cases, without inventing additional semantic scopes. Function scopes, activation records and retained captures are described below.

Temporaries start at `t0`. Allocation chooses the smallest available numeric slot; release occurs only once the final consuming instruction is emitted. Every occurrence retains its own type, so a free slot may be reused for a different type. Live operands cannot overlap; repeated statements do not reserve fresh slots indefinitely. `peakTemporaries` records peak live demand, not physical register usage, stack size, or a claim of global optimality.

Direct functions and abstract activation records

Signatures, binding and validation

Named functions support typed numeric/boolean/string/class/list contracts and guarded untyped formals. Results follow their declared semantic contract; the analyzer's result remains `null` when omitted. For example:

```
function increment(value: integer): integer {
    value = value + 1;
    return value;
}
let answer = increment(2);
```

Calls bind through the active lexical environment, never by a name-only registry lookup. Inferred `let/const` function aliases and mutable alias assignments retain function values; an alias shadowing a function name is callable. Non-function shadowing remains invalid. The existing grammar uses identifier callees; no lambda or function-type annotation syntax is added. Untyped parameters use guarded tagged values; bodyless external AST functions and malformed signatures are explicitly rejected. A function uses the existing semantic function scope spanning its whole declaration; its parameters and direct body declarations share that scope. Real nested blocks and for scopes retain their own semantic IDs. Parameters are mutable invocation-local values, not aliases to caller symbols.

Every body is lowered once in source order, including uncalled and unreachable code, with an independent existing temporary pool and a fresh transfer stack. All direct call edges are collected without recursively lowering callees. Self and mutual recursion use the same invocation-local frames and explicit return continuations as other calls. Uncalled cycles are valid, but every body still undergoes static validation. Compilation does not guarantee runtime termination.

Scalar returns use existing type compatibility and explicit `i2f` widening; there is no synthesized scalar default. Semantic missing-return diagnostics remain authoritative. Null-result procedures accept bare return, return-null (also grouped null), and fallthrough; fallthrough emits an explicit no-value return. Null-result expression calls execute exactly once, with no destination or result-frame cell, then yield the literal `null`. A null return expression is evaluated for its effects before the no-value return. Omitted result annotations still mean null, not inferred return types.

Evaluation and global initialization

Operands and arguments evaluate left to right, once. Before evaluating a later effectful subtree (including assignments), an earlier symbol operand is copied into a reserved temporary. This applies to arithmetic, comparisons, argument lists and a switch selector before effectful case tests. Literals and already-live temporaries need no copy. For `let g=1; ... let x=g+bump();`, where `bump` increments and returns `g`, `x` is 3, not 4. Similarly, `pair(g,bump())` receives 1 and 2, not 2 and 2. Widening occurs before later arguments are evaluated. Logical `&&`/`||` retain runtime short-circuit skipping while both operands are statically validated. Programs without calls or assignment effects receive no extra snapshot instructions.

Function bodies retain declaration-time global visibility and switch-blocked names, not the final variable inventory. Global reads and writes are allowed when initialized; the absence of a semantic `Captured` flag on globals does not provide use-to-binding edges. Lowering resolves actual lexical uses by identity. Each procedure records direct global references, then inherits its callees' dependencies transitively by repeatedly unioning dependency sets until none grows. This finite monotone fixed point also covers cyclic call graphs, including dependencies reached through outgoing edges of a cycle. Each main call records the initialized global bindings available at its own call site. After graph resolution, a missing binding rejects compilation with `call requires initialized global binding s<scope>.<name>` at that call. Reads **and writes**, including statically skipped references, conservatively require initialization; no zero/default value is invented.

A caller's declaration need not see a callee's transitive globals: a forward callee may be declared after those globals initialize. For example, a function `first` declared before global `g` can call a later function `later` whose body reads `g`; calling `first` after `g` initializes is safe. Calling it before that initializer is rejected. A direct read of later `g` in `first` itself is still subject to the existing semantic declaration-order visibility rule.

Procedure and instruction metadata

`Program.instructions` remains the executable main tape. Function declarations do not execute inline. Optional `procedures` is appended after the legacy Program fields and omitted entirely for programs without functions. Each procedure contains, in order: `label`, `scopeId`, `span`, ordered `parameters`, `resultKind`, `frame`, `instructions`, `peakTemporaries`, and optional sorted transitive `globals`. Labels use the global symbol identity (`s1.increment`); branch labels are local to their tape. The procedure span is the declaration span, and all operations retain their originating source spans.

Instruction fields retain their old order and tags, followed by optional `callee`, `arguments` and `returnValue`. Call arguments are caller-frame operands in formal order; a scalar call has a typed temporary `target`, and a null-result call has a zero-valued unused target. A return uses `returnValue` when scalar; unused legacy target/left fields are zero-valued and are not reads/writes. Empty argument lists and absent return values are omitted in JSON.

All argument values must be read/copied **before** constructing or entering the callee frame, even when caller and callee use the same `t0` name. The caller's live cells remain reserved across calls. On return, read the result before discarding the callee, restore the caller/continuation, and write the caller's destination. Each invocation has fresh parameter/local/temporary storage; the legacy direct-call path shares only the global store. Calls pass scalars by value; closure programs additionally retain shared lexical cells as described below.

Cell layout and text rendering

`frame.cells` is the total cell count. `frame.slots` lists contiguous zero-based `offset`, `size: 1`, `role`, optional owned `operand` and optional `dynamic` in this exact order:

Order	Role	Contents
0	`continuation`	Return address / suspended caller instruction position
1	`controlLink`	Calling activation
2	`globalAccessLink`	Shared global store only, not a closure environment
next, non-null result only	`result`	Declared result value, including reference values; absent for null results
next	`parameter`	Ordered formal operands with declared types or tagged unknown values
next	`local`	All function-owned variables/constants, including block/for locals, sorted by declaration source offset with operand-identity tie-break
last	`temporary`	`t0` through `t<peak-1>`, dynamically typed cells

All cell sizes and offsets are **abstract cells**, not bytes, machine words, physical registers or a completed target ABI. Parameter/local operand names encode semantic scope ownership. A temporary's slot operand has an empty type and `dynamic: true`: its individual instruction occurrences carry their types, because the existing pool can reuse a cell first as integer, then boolean or float. No second allocator is introduced. Main does not acquire fictitious procedure-frame metadata.

With procedures present, text begins `main:` and then prints main instructions, followed by `procedure <label>(<formals>) -> <result> [scope=<id> frame=<n> cells]:`, one `cell <offset> <role> [operand] [dynamic]` line per slot, the procedure instructions, and `end <label>`. Without procedures, old text and JSON (including field ordering, temporary numbering and layout absence) are unchanged.

`functions.cps` terminates without print and would compute `total = 7` and `result = 8.0`. The CLI emits its TAC, not execution output. Bounded test-only frame execution checks independent invocation storage, continuation/control restoration, global effects, defined slots, ownership and label/step/depth limits.

Closures and retained lexical cells

`closures.cps` installs a counter into a global inferred function alias. After `install(10)` returns, calling the saved function computes `first = 11` and `second = 12`. Copying that alias preserves the same counter. Calling `install` again creates a different counter; sibling closures from one installation share one cell. The CLI generates code only; executable proof uses the bounded test oracle.

Explicit IR contract

Operation/metadata	Contract
`Program.environments`	Optional layouts sorted by semantic `scopeId`, with `parentId` and identity-sorted typed `bindings`. A layout is not a live allocation.
`enterEnvironment`, `environment`	Allocate fresh undefined cells every time this instruction executes. Link the actual currently active lexical parent instance. Function parameters enter initialized from the existing argument frame cells; global cells use the shared global store.
`makeClosure`, `callee`, `environment`, `target`	Construct a `function` value pairing the procedure identity with the exact retained defining environment instance. Does not read captures or initialize them.
`callIndirect`, `left`, `arguments`, `signature`, `target`	Read the selected callable and arguments, check its full expected signature (including nominal/list contracts) and initialization requirements, then invoke using its retained lexical parent. Guarded unknown calls use the actual callable contract as described in T10.

Operation/metadata	Contract
`Procedure.lexicalScope`	Defining environment layout required by the procedure. Its own function scope is entered on each invocation.
`Procedure.captures`	Sorted transitive non-global binding requirements, including reads and writes. Requirements owned by a caller are checked at its call site rather than propagated outside that activation.

Text includes environment layout declarations and explicit environment-entry, closure-construction and indirect-call instructions. Optional JSON fields are omitted for programs using only previously supported direct functions. Existing direct-call text/JSON, frame offsets and temporary numbering remain unchanged.

In a closure program, environment cells are authoritative storage for lexical symbols; parameter/local frame slots remain the ownership inventory, with arguments staged in parameter slots before entry. Temporaries/results/control cells retain the existing allocator and frame behavior. This is an abstract storage contract, not a target ABI or a garbage collector. Retention must outlive a defining return; reentering a block, including a loop body, allocates a new environment instance. Grandparent captures follow retained parent links. No block-exit opcode is needed: operands and closure construction name static layout identities, and new entries replace only the active instance, never the instances already retained by closures.

Initialization and signature safety

Function declarations are hoisted; variable initialization is not. Creating an alias of a hoisted closure before its captured variable initializes is permitted; calling it requires initialized cells. Known direct/nested call edges use the existing fixed-point global requirements plus lexical capture requirements. An unsafe statically known nested call rejects atomically with `call requires initialized captured binding s<scope>.<name>`. Uncalled and unreachable nested bodies are still validated in full.

Aliases can be reassigned dynamically. Analyze currently compares function kinds, not full function signatures, on assignment; this unit deliberately does not change that semantic API. A typed indirect call therefore carries the statically expected `signature`; a backend must reject mismatched actual parameter kinds, arity or result kind **before invoking**. The test oracle reports `indirect call signature mismatch`. It must also check the actual procedure's transitive `globals` and `captures` against its retained environment before invoking, including write-only requirements. Undefined cells fault rather than inventing a default. Thus a dynamically selected early global/capture call can compile but fault at execution; it is not a new static definite-initialization guarantee. An indirect call reached through another direct or recursive call keeps the same guard. Static diagnostics still discard the entire TAC, including invalid alias initializers and failed static scope/initialization guards elsewhere in the function graph.

The callee is selected and frozen before arguments evaluate. Arguments and earlier scalar operands remain frozen across effectful calls using the existing temporary pool. Copying a function copies its procedure/environment pair, not captured scalar snapshots. Arithmetic/comparisons retain explicit type checks, with guarded unknown operations described below. Function results follow their declared type, including reference contracts, with omitted results still `null`; there are no function-type return annotations, inferred closure returns or new lambdas.

Conditions and control flow

Comparisons evaluate left to right and produce booleans; mixed numeric operands receive explicit integer-to-float conversions. Numeric arithmetic requires numeric operands, not booleans; string addition has its own contract. Boolean literals, initialized declarations/constants, identifier assignments, expression statements and unary `!` are supported.

`a && b` branches to `b` only when `a` is true; `a || b` branches to `b` only when `a` is false. Grouping

and nested negation retain these semantics. A logical expression used as a value reserves a live result temporary before lowering its operands, then writes true or false on the corresponding incoming edge before joining. Recursive RHS operations cannot recycle that reserved result. A condition can instead branch directly without materializing a logical value.

`if/else` lowers both bodies but executes only the selected body. Nested `if`, `else if`, no-else, empty and unbraced bodies use the frontend's existing block wrappers and matching semantic child scopes. Each declaration initializer reads outer bindings before activating its own name; branch-local names do not escape. Both sides of a logical expression and both branches are always statically validated, even after a literal condition. Unsupported syntax or semantic errors in unreachable source still reject the entire compilation.

The conditional example guards division by zero and selects `quotient = -1`. The CLI emits TAC only; execution assertions use a bounded test-only interpreter, not a product runtime or optimizer.

Source-compiler boundary

Loops and transfers

Construct	Execution order	Nearest-loop `continue` destination
`while`	Condition, body, repeat	Condition
`do-while`	Body, condition, repeat if true	Condition
`for`	Initializer once, condition, body, post, repeat	Post, then condition

`break` exits the nearest loop or switch, skipping a for post when exiting that loop. An omitted for condition is true; initializer and post are optional independently. Nested and consecutive loops retain separate transfer targets. All bodies and posts are statically validated even when a literal condition prevents execution; rejection emits no TAC.

A for initializer uses the existing semantic loop scope, with a child scope for the body. While/do bodies use their block scopes; a do condition cannot see body locals. Shadowing initializers read outer bindings before activating their names. Header/body/post temporaries are consumed before backedges; runtime iterations reuse emitted temporary identities without increasing static allocation demand.

A whole for post can be a simple identifier assignment expression, optionally grouped: `for (;; (i=i+1)) { if(i>=3) break; }`. Its previous store path and IR remain unchanged. T9 additionally supports value-producing identifier/property/index assignments in expression positions accepted by the existing grammar.

The frontend now identifies condition/post by their actual header delimiters, not expression count. Declaration/assignment initializers consume their own first semicolon. The intentional compatibility exception is omitted-condition, present-post headers, for every initializer variant: `for (;; i=i+1) { ... }` now has no condition and an assignment post, rather than an assignment condition that spuriously reports `SEM_CONDITION` / `condition must be boolean`. This also corrects public AST/analysis reports for that family. Grammar, AST schema and semantic policies are unchanged; other existing analysis reports are preserved.

The terminating loop example computes `total = 13` without print or a product runtime. It exercises while-continue, do-while and for break/continue.

Boolean switches

The selector must be boolean (checked at runtime for unknown values); cases retain the analyzer's compatible-type and statically unique-case rules. Numeric selectors remain semantic errors, as required by the team policy in `../entrega1/compliance.md`. The selector is evaluated once, even for an empty

or default-only switch. Case expressions execute in source order until the first match; default runs only if none match. Every selected body exits implicitly, including an empty case: there is no fallthrough.

Dispatch precedes all body labels. A selector temporary stays live and unchanged through every comparison; each comparison uses a separate boolean result slot. All tests and bodies are lowered and validated, including unselected source. Any error discards the whole TAC program. Switch dispatch itself uses the existing opcodes; its case expressions may now contain scalar direct calls.

Switch and loop frames share one transfer stack. `break` exits its nearest frame; `continue` skips switch frames to the nearest loop's condition or for-post label. A switch alone does not make `continue` legal.

Case statements share the incoming **semantic** scope and symbol IDs; only real blocks create child scopes. TAC conservatively requires shared bindings to be initialized before the switch. A case may initialize and use its own local declaration, but later cases and code after the switch cannot read or assign that case-only binding. This restriction also applies to default-only switches and to declarations inside nested switches sharing the same scope. Such uses report `TAC_UNSUPPORTED` in phase `codegen` with the located message `switch binding is not guaranteed initialized on this path`.

This is a code-generation capability restriction, not a change to language scope or analysis reports. Blocked names cannot silently fall back to an outer binding shadowed by an earlier case declaration. A declaration's own initializer still reads the outer name before activation. Initialized outer bindings remain readable and writable across cases and nested blocks; no default values are invented.

The switch example computes `total = 6` without print. It demonstrates ordered dispatch, implicit exit, switch break and continue through a switch to a for post.

Objects, bound methods and strings

Generate the print-free object example with either CLI format:

```
go run ./cmd/compiscript-tac testdata/compiscript/tac/objects.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/objects.cps
```

The bounded test oracle executes the emitted JSON to `observed = 9`, `first = 11`, `second = 12`, `independent = 21`, `shared = 30`, and `label = "counter:ready"`. The CLI generates TAC; it is not an interpreter.

Allocation and initialization

Each allocation has reference identity and its own initially undefined field cells. Copies, class parameters/results, field stores, and subtype assignments retain that identity; they do not copy the instance's fields. Equality compares object identity. A null class reference can be stored and compared, but member access faults with `null receiver` in the oracle.

Construction evaluates arguments once, left-to-right, and snapshots them across initializer calls. Then **all inherited and own field initializers run base-to-derived, in declaration order within each class**. Only afterward does the selected nearest constructor run once. A subclass without its own constructor uses the nearest inherited constructor; a subclass with its own constructor does not automatically call any ancestor constructor. This ordering is the explicit S3 user decision, not a default borrowed from JavaScript or TypeScript.

A field without an initializer remains undefined until written. Reading it faults with `undefined field`; no zero, empty string, or null default is fabricated. Constant field initializers can write their own field exactly once; ordinary mutability and member access remain governed by Analyze. Initializers and methods are validated even when their class is unused. Unsupported code rejects the entire compilation.

Explicit object IR

Operation/metadata	Contract
`Program.classes`	Optional class layouts in semantic-scope order. Each declaration has a scope-qualified `id`, name, parent identity, lexical scope, source span, field layout, virtual method table and initializer procedure.
Field layout	Inherited fields retain their declaration identity, owner and offset. Own fields append in declaration order. Cells carry full semantic types, optional nominal class identities, mutability and source spans. Offsets count abstract cells, not bytes.
`newObject`, `class`, `target`	Allocate fresh undefined field cells and retain the actual defining lexical environments for the class hierarchy. Allocation identity is distinct from static layout identity.
`loadField`, `left`, `class`, `member`, `offset`, `target`	Read one field from the selected receiver after validating nominal ownership and layout identity.
`storeField`, `left`, `right`, `class`, `member`, `offset`	Read receiver and value before mutation. The receiver is retained across RHS calls; no receiver expression is evaluated again.
`bindMethod`, `left`, `class`, `member`, `target`	Select the implementation from the receiver's actual class and pair it with that receiver and its defining environment. Selection happens before argument evaluation.
`bindMethod`, `callee`	Internal initializer selection names the exact initializer procedure instead of virtual dispatch. Construction emits one initializer call per class, then at most one constructor call.
`Procedure.receiver`	Optional implicit receiver cell, separate from explicit source parameters. It enters the same frame/environment infrastructure as parameters. Nested functions capture that lexical cell, not a caller's current receiver.
Operand `class`, `Procedure.resultClass`, call `typeClasses`	Optional scope-qualified nominal identities for class values and full indirect-call signature checks. An indirect call checks parameter/result identities as well as kinds and arity before invocation.

Bound methods use existing `callIndirect`, frames, results, initialization guards, retained environments and temporary allocation. Escaped nested functions keep their lexical `this`; bound method aliases keep the receiver selected when the alias was created. Unqualified field/method names use the analyzer's lexical class scope: inherited property access remains available through `this`, but an inherited field is not inserted as a new unqualified lexical binding.

Strings support literals, storage, typed parameters/results, concatenation and equality/inequality. String and class storage compatibility is separate from numeric arithmetic; references are not accepted as arbitrary numeric operands. Nominal class metadata is omitted when unused, preserving prior supported TAC text/JSON. The oracle rejects malformed class identities, inheritance cycles, field offsets/owners, missing initializers/methods and incompatible dynamic calls. These guards document an abstract backend contract, not new static diagnostics.

Inferred aliases retain the initializer's scope-qualified class identity even when a nested class shadows its textual type name. The same identity is recorded in environment/frame storage; field reads and function results retain their declaring origins. Explicit annotations and existing typed stores still enforce their declared nominal identities. This uses private lowering metadata rather than changing Analyze or introducing another frontend binding model.

Lists, indexing and foreach

Lists are fixed-length references, not copied arrays. Assignment, arguments, results, fields and nested elements retain reference identity; `==`/`!=` compare identities. Each execution of a literal allocates a distinct reference, even for `[]`. Contextual empty, nested and ragged literals use the existing semantic element types. Scalar arithmetic validation is unchanged. No source `.length` member or append API exists.

Instruction	Operands	Contract
`newList`	`target`, ordered `arguments`	Read element values, then allocate a fresh reference
`listLength`	`target`, `left`	Read the reference's internal length into an integer
`loadIndex`	`target`, `left`, `index`	Bounds-check the integer index and read one element
`storeIndex`	`left`, `index`, `right`	Bounds/type-check and replace one element without resizing

Operand `shape` records the complete slash-separated element-kind chain: an `integer[][]` operand has `type: "list"`, `shape: "list/integer"`. Nominal leaves also retain their declaration-qualified `class` identity, including under shadowing. Fields and procedure results carry matching optional `shape/resultShape` metadata; indirect signatures check the complete nested shape and nominal origins before entering a callee. These fields are omitted from programs that do not use lists. Function-valued list elements also retain their original parameter/result nominal identities through nested indexing, inferred aliases and foreach bindings. Extracting a function never re-resolves its signature's class names in the extraction scope; the existing indirect-call guard still rejects incompatible replacements before executing the selected function body.

Literal elements and collection/index/RHS expressions evaluate once, left to right. List and index operands are frozen before later effects can reassign their source bindings. Reads precede destination writes; live values use the existing temporary pool and invocation frames, including recursive callers. A store checks its target bounds after evaluating its RHS, so an RHS effect may precede a runtime bounds fault.

Foreach evaluates its iterable once and retains that initial reference and internal length. This is the practical fixed-length policy: no source operation can resize a list. Reassigning the source binding does not redirect iteration; mutations of the retained list are visible to subsequent entries. Each real body entry enters its existing semantic block scope, initializes a fresh mutable iteration binding and then executes the body. Assigning that binding does not replace the element itself; mutating an object/list reference through it still affects aliases. Escaping closures retain distinct entry cells. Continue advances the private index; break exits; nested loops/switches reuse the existing nearest-transfer stack.

Semantic const-root, type and static-bounds diagnostics remain authoritative and discard the entire TAC report. Dynamic null/bounds/type faults are IR backend obligations, exercised only by the bounded test oracle, not new Analyze diagnostics. The analyzer permits covariant list views (including numeric widening); the oracle widens integer element reads through a float view and guards incompatible later reads after alias mutation. It never silently reinterprets an incompatible nominal object or nested shape. Lists are not a product execution engine or a target ABI.

`collections.cps` exercises shared nested lists and captured iteration cells. Its CLI-emitted JSON executes in tests to `shared = 9`, `sum = 12`, `first = 3`, `second = 6`, `continued = 4`; the CLI itself emits code, not those values.

Supported source

The implemented source includes print, `%`, lazy ternaries, value-producing assignments, typed/untyped uninitialized declarations and null values. Unknown values use explicit guarded instructions rather than fabricated static types. Malformed ASTs, missing scope identities and actual diagnostics reject atomically. No partial TAC may escape after lexical, syntax, semantic, or lowering errors. Original source bytes are preserved; Compile retains the matching Analyze diagnostics/scopes. Numeric expression statements are supported; their final temporary is released without a store. Null can occupy binding, field and list-element cells as well as typed reference storage. Unannotated fields retain the analyzer's unknown contract in their private layouts and every inherited copy. An initializer, including null or an outer-alias call, supplies a value without narrowing the field's storage type. Absent initializers leave fields undefined; only explicit annotations constrain their actual tags. Initializers still execute once

under constructor policy A, and semantic metadata is not rewritten. Unknown storage/untyped signatures and try/catch use the T10 contract below. Transfers without an enclosing eligible construct are rejected. Recovery nodes or violated lowering invariants are not silently skipped. Defensive declaration rejection retains the historical message `only integer, float and boolean declarations are supported`; it is not used to reject supported null, reference or unknown declarations merely for their kind. Other defensive diagnostics retain their historical numeric wording for compatibility.

Static values and effects (T9)

`let x=1; let y=x+(x=2);` computes `y=3`, `x=2`: earlier operands, arguments, callees, receivers and indices keep their source-position values across later effects. Assignments yield their stored type/value, including float widening. Ternaries reserve a join temporary before lowering either arm, evaluate the condition once and execute only the selected arm. Both arms are still validated. Expected nested-list shape and nominal/signature origins propagate through joins. `branchTrue` requires a boolean at runtime; there is no numeric truthiness. Incompatible/unknown arm types use a tagged join cell and checked typed boundaries.

`declare` resets the binding to undefined each time its declaration executes. Frame/environment reentry never inherits a previous value. Reading an undefined cell faults in the bounded oracle. An activated undefined cell satisfies existing dependency/type guards, permitting initialization through a call; an actual read still faults. Bindings not yet activated remain unavailable. Declaration activation is distinct from a default write, and the real reserved cell is inventoried by `codegen`, even when the source never assigns it. Existing call-dependency checks remain intact.

Remainder uses Go `math.Mod` semantics: `x-trunc(x/y)*y`, with the dividend's sign. Two integers produce integer remainder; float/mixed operands produce float after explicit widening. A zero divisor faults before a destination write. This is the practical KISS extension of existing numeric promotion, not Euclidean modulo.

Print uses `left` and a source span, no destination and no synthetic call. The CLI generates TAC; it does not execute print. The test-only oracle shares one output collector across frames and appends a newline per print. Strings are raw; booleans are `true`/`false`; null is `null`; numbers use shortest round-trippable binary64 text. References use `<object s1.A>` / `<function s1.f>`, never physical addresses. Lists use `[1, 2]`, with active-cycle detection, a 64-value budget and depth limit 8 (`<cycle>` / `...`). This deterministic, bounded representation avoids a new runtime or serialization dependency; it is not a product interpreter.

Guarded values and synchronous exceptions (T10)

The semantic `error` kind is an absent-information placeholder, **not** a new source `any` type or a claim that invalid executions are well typed. An `error` operand stores the actual runtime tag/reference. Untyped formals, aliases, captures, member/index results and joins retain it. Existing statically typed programs need no new metadata: `guarded` is omitted unless used.

Instruction / metadata	Contract
Arithmetic/comparison/unary instruction with `guarded:true`	Check actual operands; numeric promotion, numeric comparisons and string addition when valid; strict boolean negation; invalid operands fault
`checkValue target = left`	Validate the actual value before a typed boundary; integer-to-float widening is permitted; nominal origins and list rank/leaf contracts remain checked
`callIndirect` with `guarded:true` and no `signature`	Select the actual callable once; check its complete actual formal contract before body effects; use its actual result contract
`dynamicMember` / `dynamicStore`	Resolve fields/bound methods on the actual object; preserve owner, mutability and stored-value type; missing/null/non-object receivers fault
`pushHandler label` / `popHandler`	Register/remove a handler in the current dynamic invocation, not in a retained lexical environment

Instruction / metadata	Contract
`catchException target`	Consume the pending opaque `exception` reference into a fresh catch scope; the selected handler has already been removed

The existing `copy`, list and branch operations retain their explicit checks. Unknown cells have no invented initialization default. Conditions never use truthiness. Empty/generic list references retain allocation identity; a typed use checks a view/contents without copying the container or silently changing its constraints. Known incompatible typed list contracts remain strict. Existing typed covariant-view mutation/fault timing is unchanged.

Missing function result annotations still mean **null**, not inferred results. Unknown non-null returns from these functions fault before handler cleanup. Null remains null through typed reference views and unknown formals. Guarded equality permits numeric promotion and null/reference identity comparisons; incompatible non-null runtime tags fault rather than inventing coercions. Dynamic null-result calls execute once and write null to the caller's tagged destination without adding a callee result cell. Actual callable declaration, receiver, lexical environment and full nominal signature remain attached to the reference through unknown storage and collections.

KISS exception policy: explicit language faults (undefined/null access, bounds, zero division/remainder, invalid dynamic type/callee/member/index/condition and runtime nominal/signature guards) are catchable. Their messages remain stable; the bounded oracle prints an exception's message without physical addresses or internal stack traces. Each delivered exception has opaque reference identity. Instruction and catch-binding source spans remain in IR; messages do not invent host stack locations. Effects before the fault are committed, not rolled back.

The nearest handler on the **dynamic call stack** wins, abandoning intervening callee continuations. Escaped closure cells from abandoned calls remain retained. Catch faults reach an outer handler because the selected handler is removed before catch-body entry. A normal try path skips catch. Returning validates and reads its value **before** cleanup; escaping return/break/continue remove only crossed handlers, while transfers staying inside a try retain its handler. New calls do not inherit handlers from their closure's defining environment.

Malformed IR, resource-budget failures and host errors are not language exceptions. The oracle preflights instruction/frame/layout contracts and uses explicit fault types, never message-substring classification or blanket panic recovery. It remains test-only: 10,000 instructions, 64 call levels and bounded frame sizes. The CLI/API generate TAC; they do not execute this oracle.

Every valid AST kind has a lowering dispatch: statements Block, VarDecl, ConstDecl, Assign, FunctionDecl, ClassDecl, Expr, Print, If, While, DoWhile, For, Foreach, TryCatch, Switch, Break, Continue and Return; expressions Identifier, Literal, Array, This, New, Group, Unary, Binary, Ternary, Assign, PropertyAssign, Call, Index and PropertyAccess. `BadStmt/BadExpr` are recovery nodes and reject. This kind inventory is not a guarantee that every accepted execution succeeds: dynamic faults and conservative initialization guards remain intentional.

Library quick path

Inside this Go module, call `compiscript.Compile(source)` from `genanalex/internal/compiscript`. The result is a `CompilationReport` with JSON fields `tac`, `diagnostics`, `scopes` and optional successful `codegen`. Parse and semantic analysis each run once. Any frontend or semantic error preserves the same ordered diagnostics and scope snapshots as `Analyze`, sets `tac` to `null`, and skips lowering entirely. Lowering reports a failed static initialization/scope guard or invariant with code `TAC_UNSUPPORTED`, phase `codegen`, a descriptive message, and its source span; `tac` is also `null`, even when earlier statements were supported. An empty source succeeds with `instructions: []`, `peakTemporaries: 0`, and `diagnostics: []`.

For example, `let x=1/2; x=x+3;` produces exactly:

```
t0:float = i2f 1:integer
t1:float = i2f 2:integer
t0:float = t0:float / t1:float
s1.x:float = t0:float
t0:float = i2f 3:integer
t0:float = s1.x:float + t0:float
s1.x:float = t0:float
```

The peak is two temporaries: division consumes `t0` and `t1` before reusing `t0` as its destination. Each JSON instruction also retains its source span and operand kinds. No arithmetic is evaluated by the compiler.

For `let x=1; print(x);`, the result includes `print s1.x:integer`. For `let x:integer;`, it emits `declare s1.x:integer` without a fabricated value. For `let x=missing;`, the existing semantic diagnostic is returned instead of attempting code generation. The default command renders the same TAC; JSON exposes the complete report.

Remaining boundaries

- Conservative initialization guards for switch/global/captured bindings remain intentional; this delivery does not replace them with a stronger path analysis.
- Target-machine byte sizes/offsets and calling ABI, garbage collection and target-specific lexical links. Abstract retained lexical environments and function parameter/local/temporary/result/control cells are implemented, not this target layout.
- The original construction gap is resolved by T9/T10; no new concrete normative implementation gap is established by the final report. Human/evaluation checks remain in the report checklist.
- Source-to-TAC IDE output, static main/global layouts and mixed successful/failed examples are implemented; real-browser visual evaluation remains pending.

Future memory access needs distinct opcodes and explicit operands, as direct calls already have; operations must not be smuggled into numeric symbol strings. Object and collection references use the explicit value/storage contract; they are not encoded as scalar operand strings. The missing-loop-body panic was corrected with a guarded block lookup; the acceptance battery checks located, atomic rejection without blanket recovery. This is not proof for all malformed input.

Verification

```
GOPROXY=off GOSUMDB=off GOTOOLCHAIN=local go test ./internal/compiscript ./internal/compiscript/tac
-count=1
GOPROXY=off GOSUMDB=off GOTOOLCHAIN=local go vet ./internal/compiscript ./internal/compiscript/tac
GOPROXY=off GOSUMDB=off GOTOOLCHAIN=local go test ./cmd/compiscript-tac -count=1
```

Current tests pin instruction formatting, deterministic program output, live temporary uniqueness, release protection, type-changing reuse, and bounded allocation across repeated expressions. Source-level tests pin precedence, associativity, widening, division, lexical binding, exact typed/source-located instructions, rejection without partial output, deterministic JSON, input preservation, and compatibility with analysis diagnostics. CLI tests execute the built binary for exact TAC/report output, repeated runs, located rejection, exit codes, argument/input errors, and unchanged files. The construction inventory and passing cases are not exhaustive correctness or measured line coverage. Conditional tests execute generated instructions with a finite step budget, validate labels/targets and defined temporaries, and check skipped/demanded division traps, truth tables, nested joins, branch scopes, source preservation and deterministic bounded temporary allocation. Loop tests use the same bounded interpreter for optional clauses, backedges, nearest transfers, scope shadowing, numeric promotion and short-circuit traps. Switch tests reuse that interpreter for dispatch order, selector lifetime, skipped traps, implicit exit, nested transfers, case initialization restrictions and bounded temporary pressure. CLI tests cover text/JSON success and atomic skipped-body rejection. Function tests execute separate tapes with fresh abstract frames, caller-value freezing, mutable by-value parameters, widening, null/scalar returns, nested control flow and transitive global effects. They pin deterministic layouts/JSON/text, dynamic temporary cells, immutable source/AST/scopes and atomic function/graph/ initializa-

tion rejection, including uncalled bodies. The prior condition oracle is unchanged; the frame oracle is test-only, with instruction/depth limits rather than a product runtime overflow policy. Recursive regressions exercise factorial, multiple live recursive results, mutual boolean calls, null-result procedures, cyclic global dependencies and initialization rejection. `recursion.cps` would compute `product = 1 20` and `sequence = 21`; the CLI emits TAC, not those values. Closure tests additionally execute escaped counters, distinct invocations, shared sibling cells, recursive owners, grandparent captures, block/loop reentry, hoisting, shadow initializers and callee-selection order. Built CLI JSON is decoded and executed through the same bounded retained-environment oracle. Invalid layouts, undefined/out-of-bounds environment cells, signature mismatches and initialization faults are checked without adding a product interpreter.